

JavaScript

Complete End-to-End Developer Guide

Basics to Advanced — Why, What, When & How for Every Concept,
Real-Time Examples, Bugs, Real Issues, Fixes & Interview Q&A

Table of Contents

1. Why JavaScript Exists — The Third Layer of the Web
2. How JavaScript Actually Runs — Engines, Runtime & the Browser
3. Variables — var vs let vs const Deep Dive
4. Data Types — Primitives vs Reference Types
5. Type Coercion & Equality — == vs ===
6. Operators Deep Dive
7. Control Flow — if/else, switch, Ternary
8. Loops Deep Dive — for, while, for...of, for...in
9. Functions Deep Dive — Declarations, Expressions, Arrow Functions
10. Function Parameters — Default & Rest Parameters
11. Scope Deep Dive — Global, Function & Block Scope
12. Hoisting Deep Dive
13. Closures Deep Dive — Why, What & When
14. The "this" Keyword Deep Dive
15. Arrays Deep Dive & Common Methods
16. Array Higher-Order Methods — map, filter, reduce
17. Objects Deep Dive
18. Object Methods & Property Shorthand
19. Destructuring — Array & Object
20. Spread & Rest Operators Deep Dive
21. Template Literals
22. ES6+ Feature Recap Table
23. Prototypes & Prototypal Inheritance Deep Dive
24. Classes in JavaScript
25. Class Inheritance, super & Static Members
26. Getters & Setters
27. The Event Loop Deep Dive — Call Stack, Task Queue, Microtask Queue
28. Callbacks & Callback Hell
29. Promises Deep Dive
30. Async/Await Deep Dive

31. Error Handling — try/catch & Custom Errors
32. Fetch API & Making HTTP Requests
33. JSON Deep Dive
34. The DOM Deep Dive — Selecting & Manipulating Elements
35. Events Deep Dive — Bubbling, Capturing & Delegation
36. Forms & Events in JavaScript
37. localStorage & sessionStorage in JavaScript
38. Debouncing & Throttling
39. Modules — import/export Deep Dive
40. Generators & Iterators
41. Map & Set Deep Dive
42. Regular Expressions Basics
43. Memory Management & Garbage Collection
44. Memory Leaks — Common Causes & Fixes
45. Strict Mode
46. Immutability & Pure Functions
47. Functional Programming Concepts in JavaScript
48. Common Design Patterns in JavaScript
49. Performance Optimization Tips
50. NPM & package.json Deep Dive
51. Modern Build Tools — Webpack & Vite Overview
52. Testing JavaScript — Jest Basics
53. JavaScript in a Full-Stack World (React + Spring Boot)
54. Common JavaScript Bugs, Issues & Fixes (Extended)
55. JavaScript Interview Questions & Answers (40 Q&A)
56. Quick Revision Summary

1. Why JavaScript Exists — The Third Layer of the Web

By 1995, HTML could describe structure and CSS could describe appearance, but web pages were still completely **STATIC** — nothing could react to a user's click, validate a form before submission, or update content without a full page reload from the server. Netscape hired Brendan Eich, who created JavaScript in just **10 days** to give web pages actual **BEHAVIOR** — the ability to respond to events and manipulate the page after it loaded.



What problem did JavaScript actually solve?

- **Instant feedback** — validating a form field the moment a user types, without waiting for a server round-trip.
- **Dynamic content** — updating part of a page (a shopping cart count, a live chat message) without reloading the entire page.
- **Interactivity** — dropdown menus, image carousels, drag-and-drop, and everything else that makes a page feel like an application rather than a document.

Important distinction: JavaScript (the language covered in this guide) has **NOTHING** to do with Java — the similar name was purely a 1995 marketing decision by Netscape to ride Java's popularity. They are two completely unrelated languages with different syntax, purpose, and runtime.

Real-time example: An e-commerce checkout form uses JavaScript to instantly show "Passwords don't match" the moment you type a mismatched confirmation password — no page reload, no server call, pure client-side behavior — exactly the kind of interactivity HTML/CSS alone could never provide.

2. How JavaScript Actually Runs — Engines, Runtime & the Browser

Component	Role
JavaScript Engine (e.g., V8 in Chrome/Node.js)	Parses and actually EXECUTES JavaScript code — converts it to machine code for speed
Browser Runtime / Node.js Runtime	Provides extra capabilities the engine itself doesn't have: DOM APIs (browser), file system APIs (Node.js), timers (setTimeout), networking (fetch)
Call Stack	Tracks which function is currently executing (Section 27 covers this in depth)
Web APIs / Node APIs	Browser/Node-provided features like <code>setTimeout</code> , <code>fetch</code> , and DOM events that JavaScript itself doesn't define but can call into

Key insight: JavaScript the LANGUAGE is the same everywhere (syntax, closures, promises). What's DIFFERENT between running it in a browser vs Node.js is the surrounding RUNTIME's extra APIs — a browser gives you `document` /DOM access; Node.js gives you file system and OS-level access instead. This is why browser JavaScript can't read local files directly, but Node.js can.

3. Variables — var vs let vs const Deep Dive

```
var oldWay = "function-scoped, hoisted, can be redeclared"; // legacy, avoid in new code
let mutable = "block-scoped, can be reassigned";
const fixed = "block-scoped, CANNOT be reassigned";

fixed = "new value"; // TypeError: Assignment to constant variable.
```

Keyword	Scope	Reassignable?	Redeclarable?	Hoisting Behavior
var	Function-scoped (ignores blocks entirely)	Yes	Yes (no error)	Hoisted & initialized as <code>undefined</code>
let	Block-scoped (<code>{ }</code>)	Yes	No (SyntaxError)	Hoisted but NOT initialized ("Temporal Dead Zone")
const	Block-scoped	No	No	Hoisted but NOT initialized (TDZ)

const does NOT mean "immutable" — it only prevents REASSIGNING the variable itself. An object or array stored in a const variable can still have its properties/contents changed freely: `const arr = [1,2]; arr.push(3);` // totally fine, `arr` itself was never reassigned.

ISSUE A variable declared with `var` inside a for-loop inside a function seems to "leak" and behave unexpectedly when used inside a callback.

FIX `var` is function-scoped, not block-scoped — it doesn't create a new binding per loop iteration. Switch to `let`, which creates a genuinely fresh binding for each iteration (Section 13 shows the classic closure-in-loop bug this fixes).

Modern best practice: Use `const` by default for everything; only use `let` when you KNOW the variable must be reassigned later; avoid `var` entirely in new code.

4. Data Types — Primitives vs Reference Types

Category	Types	Behavior
Primitives	string, number, boolean, null, undefined, symbol, bigint	Copied BY VALUE — assigning or passing one copies the actual value; comparing two equal primitives with <code>===</code> returns true
Reference Types	object, array, function (technically a callable object)	Copied BY REFERENCE — assigning or passing one copies a POINTER to the same underlying data; comparing two different objects with identical content returns false

```

let a = 5;
let b = a;
b = 10;
console.log(a); // 5 — primitives copy by value, 'a' is untouched

let obj1 = { count: 1 };
let obj2 = obj1;          // obj2 points to the SAME object as obj1
obj2.count = 99;
console.log(obj1.count); // 99 — both variables reference the same object in memory

console.log({a:1} === {a:1}); // false — different objects in memory, even with identical content

```

ISSUE Updating a "copy" of an object/array elsewhere in the code unexpectedly changes the original too.

FIX Objects/arrays are copied by reference — use `{...original}` (spread, Section 20) or `structuredClone(original)` to create a genuinely independent copy before mutating it.

5. Type Coercion & Equality — `==` vs `===`

```
console.log(1 == "1");    // true - == converts types before comparing ("coercion")
console.log(1 === "1");   // false - === compares value AND type, no conversion
console.log(null == undefined); // true
console.log(null === undefined); // false
console.log(0 == false);  // true
console.log("" == false); // true
console.log(NaN === NaN); // false! NaN is never equal to anything, even itself
```

Industry-standard rule: ALWAYS use `===` and `!==` in real code, never `==` / `!=`. The `==` operator's coercion rules are notoriously confusing and inconsistent (it's a common source of subtle bugs), and virtually every linter/style guide (Airbnb, Google) enforces strict equality by default.

ISSUE Checking `if (value === NaN)` never evaluates to true, even when value genuinely is NaN.

FIX NaN is the only value in JavaScript that is never equal to itself — use `Number.isNaN(value)` instead of any equality comparison to check for NaN.

6. Operators Deep Dive

```
// Logical operators used for more than just booleans
const name = userInput || "Guest";           // fallback if userInput is falsy
const config = options ?? defaultConfig;     // nullish coalescing: fallback ONLY for
null/undefined
user?.address?.city;                         // optional chaining: safely access deep nested
properties

// Falsy values in JavaScript (everything else is truthy):
// false, 0, "", null, undefined, NaN
```

Operator	Difference from
?? (nullish coalescing)	Only falls back if the left side is exactly <code>null</code> or <code>undefined</code> — unlike <code> </code> , it does NOT fall back for other falsy values like <code>0</code> or <code>""</code> , which is often what you actually want
?. (optional chaining)	Safely returns <code>undefined</code> instead of throwing a <code>TypeError</code> when accessing a property on <code>null / undefined</code> mid-chain

ISSUE `const quantity = input.quantity || 10;` incorrectly resets a genuinely entered "0" quantity back to 10.

FIX `||` treats `0` as falsy and falls back — use `??` instead, which only falls back for `null / undefined`, correctly preserving a real `0` value.

7. Control Flow — if/else, switch, Ternary

```
if (score >= 90) { grade = "A"; }  
else if (score >= 75) { grade = "B"; }  
else { grade = "C"; }  
  
switch (day) {  
  case "Mon": case "Tue": case "Wed": case "Thu": case "Fri":  
    type = "Weekday"; break;  
  case "Sat": case "Sun":  
    type = "Weekend"; break;  
  default:  
    type = "Unknown";  
}  
  
const label = isActive ? "Active" : "Inactive"; // ternary – a compact if/else expression
```

ISSUE A `switch` statement falls through and executes MULTIPLE case blocks unintentionally.

FIX Missing a `break` statement at the end of a case — every case needs its own `break` (or intentionally group cases like the weekday example above) to prevent execution from "falling through" into the next case.

8. Loops Deep Dive — `for`, `while`, `for...of`, `for...in`

```
for (let i = 0; i < 5; i++) { console.log(i); } // classic counting loop

let i = 0;
while (i < 5) { console.log(i); i++; } // repeats while condition is true

const fruits = ["apple", "banana"];
for (const fruit of fruits) { console.log(fruit); } // iterates over VALUES (arrays,
strings, Maps, Sets)

const person = { name: "Alex", age: 30 };
for (const key in person) { console.log(key, person[key]); } // iterates over KEYS (objects)
```

Loop	Best For
for...of	Iterating VALUES of arrays, strings, Maps, Sets — the modern default choice for arrays
for...in	Iterating KEYS (property names) of plain objects — NEVER use on arrays (it iterates indices as strings and can include inherited properties)

ISSUE Using `for...in` on an array produces unexpected results or includes extra unwanted properties.

FIX `for...in` is designed for object keys, not array iteration, and can pick up inherited/enumerable properties — use `for...of` or array methods (`forEach` , `map`) for arrays instead.

9. Functions Deep Dive — Declarations, Expressions, Arrow Functions

```
// Function Declaration – hoisted entirely (can be called before its definition in the file)
function greet(name) { return `Hello, ${name}`; }

// Function Expression – NOT hoisted the same way; the variable is hoisted but not its function
// value
const greet2 = function(name) { return `Hi, ${name}`; };

// Arrow Function – shorter syntax, and crucially does NOT have its own "this" (Section 14)
const greet3 = (name) => `Hey, ${name}`;
const add = (a, b) => a + b; // implicit return for single expressions
const doStuff = () => { /* multiple statements need braces + explicit return */ return 1; };
```

Type	Hoisted?	Has its own "this"?	Can be used as a constructor (new)?
Function Declaration	Yes, fully (safe to call before definition)	Yes	Yes
Function Expression	Variable hoisted, value is not	Yes	Yes
Arrow Function	Variable hoisted, value is not	No — inherits "this" from enclosing scope	No — throws an error if used with "new"

When to prefer arrow functions: Short callbacks (array methods, event handlers where you WANT to inherit the surrounding "this") and anywhere brevity helps readability. **When to prefer regular functions:** Object methods needing their own "this", or whenever you need the function to be usable as a constructor.

10. Function Parameters — Default & Rest Parameters

```
function greet(name = "Guest") {           // default parameter – used if argument is
  omitted/undefined
  return `Hello, ${name}`;
}
greet();           // "Hello, Guest"
greet("Alex");     // "Hello, Alex"

function sum(...numbers) {                // rest parameter – collects ALL remaining args into a
  real array
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4);   // 10
```

Before ES6 default parameters, developers manually wrote `name = name || "Guest";` inside the function body — which incorrectly overrides genuinely-passed falsy values like `0` or `""`, exactly the `||` vs `??` issue from Section 6. Default parameters correctly trigger ONLY when the argument is literally omitted or explicitly `undefined`.

11. Scope Deep Dive — Global, Function & Block Scope

```
let globalVar = "accessible everywhere in this file";

function outer() {
  let functionScoped = "only accessible inside outer()";
  if (true) {
    let blockScoped = "only accessible inside this if-block";
    var notBlockScoped = "var ignores the block – accessible throughout outer()";
  }
  console.log(notBlockScoped); // works – var isn't block-scoped
  // console.log(blockScoped); // ReferenceError – let respects the block
}
```

Scope chain: When JavaScript looks up a variable, it searches the CURRENT scope first, then walks OUTWARD through each enclosing scope until it finds it (or reaches global scope and throws a `ReferenceError`). This outward-searching chain is the foundation that makes closures (Section 13) possible.

12. Hoisting Deep Dive

```
console.log(hoistedVar); // undefined (NOT an error – declaration is hoisted, value is not)
var hoistedVar = "value";

console.log(hoistedLet); // ReferenceError: Cannot access 'hoistedLet' before initialization
let hoistedLet = "value";

sayHi(); // Works! Function declarations are hoisted completely, including their body.
function sayHi() { console.log("Hi!"); }

sayBye(); // TypeError: sayBye is not a function (the variable exists but is still undefined)
var sayBye = function() { console.log("Bye!"); };
```

The Temporal Dead Zone (TDZ): `let` and `const` ARE technically hoisted (JavaScript knows they exist), but accessing them before their actual declaration line throws a `ReferenceError` rather than returning `undefined` like `var` does — this stricter behavior catches real bugs earlier and is considered an improvement, not a limitation.

ISSUE Calling a function defined with a function EXPRESSION before its line in the file throws "is not a function".

FIX Only the variable declaration is hoisted, not its assigned function value — move the function expression above its usage, or convert it to a function DECLARATION if it must be called earlier in the file.

13. Closures Deep Dive — Why, What & When

A **closure** is a function that "remembers" the variables from the scope it was CREATED in, even after that outer scope has finished executing. This is one of JavaScript's most powerful — and most commonly misunderstood — features.

```
function createCounter() {  
  let count = 0; // count is "enclosed" by the returned function  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2 – count persisted between calls, private to this counter instance!  
  
const counter2 = createCounter();  
console.log(counter2()); // 1 – a completely separate, independent "count" variable
```

Why closures matter — real use cases

- **Data privacy/encapsulation:** `count` above can't be accessed or modified directly from outside — only through the returned function, similar to a private field.
- **Function factories:** Creating specialized functions from a general template (e.g., `createMultiplier(3)` returning a "triple" function).
- **Event handlers & callbacks:** An event handler closure can still access variables from when it was set up, even though the setup function has long since returned.

The classic closure-in-loop bug

```
// BUGGY – using var  
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), 100); // prints 4, 4, 4 (NOT 1, 2, 3!)  
}  
// var is function-scoped – by the time the timeouts fire, the loop has finished and i is 4  
  
// FIXED – using let  
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), 100); // correctly prints 1, 2, 3  
}  
// let creates a FRESH binding of i for each iteration, correctly captured by each closure
```

ISSUE Setting up multiple event listeners in a loop, they all reference the SAME final value from the loop

instead of their own iteration's value.

FIX Classic `var`-in-loop closure bug — switch to `let`, which correctly creates an independent binding per loop iteration.

14. The "this" Keyword Deep Dive

```
const user = {
  name: "Alex",
  greet: function() { console.log(`Hi, I'm ${this.name}`); }
};
user.greet(); // "Hi, I'm Alex" - 'this' = the object the method was CALLED ON

const greetFn = user.greet;
greetFn(); // "Hi, I'm undefined" - 'this' is lost! Called standalone, not as user.greet()

const user2 = {
  name: "Sam",
  greetLater: function() {
    setTimeout(() => console.log(`Hi, I'm ${this.name}`), 100); // arrow function - inherits
    'this' from greetLater
  }
};
user2.greetLater(); // "Hi, I'm Sam" - works correctly because arrow functions don't have their
own 'this'
```

Call Style	What "this" Refers To
<code>obj.method()</code>	The object the method was called on (obj)
A standalone function call	<code>undefined</code> in strict mode/modules (or the global object in old non-strict scripts)
Arrow function	Whatever "this" was in the ENCLOSING scope at creation time — arrow functions never have their own "this"
<code>new Constructor()</code>	The newly created instance
<code>fn.call(obj)</code> / <code>fn.apply(obj)</code> / <code>fn.bind(obj)</code>	Explicitly set to whatever object you pass in

ISSUE Passing an object's method as a callback (e.g., to an event listener) loses its correct "this" binding.

FIX Use `.bind(this)` when passing the method as a reference (`button.onclick = obj.method.bind(obj)`), or wrap it in an arrow function (`() => obj.method()`), or simply define the method itself as an arrow function/class field if the class syntax allows it.

15. Arrays Deep Dive & Common Methods

```
const fruits = ["apple", "banana", "cherry"];
fruits.push("date");           // add to end
fruits.pop();                  // remove from end
fruits.unshift("apricot");     // add to beginning
fruits.shift();                // remove from beginning
fruits.indexOf("banana");      // 1 (or -1 if not found)
fruits.includes("cherry");     // true
fruits.slice(1, 3);            // returns a NEW array (non-mutating), doesn't change original
fruits.splice(1, 1, "kiwi");   // MUTATES original – removes/inserts elements in place
fruits.sort();                 // mutates, sorts alphabetically by default (surprising for
                                // numbers!)
[10, 2, 33].sort();            // [10, 2, 33] -> [10, 2, 33] sorted as STRINGS -> [10, 2, 33]...
                                // actually gives [10, 2, 33] wrong
[10, 2, 33].sort((a, b) => a - b); // [2, 10, 33] – correct numeric sort, always pass a
                                // comparator for numbers
```

ISSUE `[10, 2, 33].sort()` returns `[10, 2, 33]` unsorted-looking output, sorting numbers incorrectly.

FIX `Array.sort()` converts elements to STRINGS and sorts lexicographically by default — always pass an explicit comparator function `(a, b) => a - b` when sorting numbers.

Mutating vs non-mutating methods matter a lot: `push/pop/shift/unshift/splice/sort/reverse` all MODIFY the original array; `slice/map/filter/concat` return a NEW array, leaving the original untouched — mixing these up is a very common source of bugs, especially in React state management where mutating state directly breaks re-rendering.

16. Array Higher-Order Methods — map, filter, reduce

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => n * 2);           // [2, 4, 6, 8, 10] – transforms EVERY
element
const evens = numbers.filter(n => n % 2 === 0);     // [2, 4] – keeps only elements passing
the test
const sum = numbers.reduce((total, n) => total + n, 0); // 15 – collapses the array into ONE
value

// Chaining – a very common real-world pattern
const activeUserNames = users
  .filter(user => user.isActive)
  .map(user => user.name);
```

Method	Returns	When to Use
map	A new array, SAME length, transformed values	Converting data shape (e.g., objects to just their names)
filter	A new array, shorter or equal length	Selecting a subset matching a condition
reduce	A single accumulated value (any type — number, object, array)	Summing, grouping, flattening, or building any custom aggregate
forEach	Nothing (undefined) — used purely for side effects	Logging, DOM updates — NOT for building a new array (use map instead)

Real-time example: An analytics dashboard computes total revenue per region from a flat list of 10,000 orders using a single `reduce` call building a grouped object — far more declarative and readable than a manual for-loop with an accumulator object built up imperatively.

17. Objects Deep Dive

```
const user = {
  name: "Alex",
  age: 30,
  address: { city: "Mumbai" }
};

console.log(user.name);           // dot notation
console.log(user["name"]);       // bracket notation – needed for dynamic keys
const key = "age";
console.log(user[key]);          // 30 – bracket notation allows a variable as the key

delete user.age;                 // removes a property
console.log("name" in user);     // true – checks key existence
console.log(Object.keys(user));  // ["name", "address"]
console.log(Object.values(user)); // ["Alex", {city:"Mumbai"}]
console.log(Object.entries(user)); // [["name", "Alex"], ["address", {...}]]
```

Use bracket notation whenever the property name is stored in a VARIABLE or contains characters that aren't valid in dot notation (spaces, starting with a number) — dot notation only works with literal, valid identifier names known at the time you write the code.

18. Object Methods & Property Shorthand

```
const name = "Alex", age = 30;
const user = { name, age };           // shorthand – same as { name: name, age: age }

const userService = {
  findById(id) { return db.find(id); }, // method shorthand – same as findById: function(id)
  {...}
};

const merged = Object.assign({}, defaults, userOverrides); // shallow merge (spread, Section 20, is now more common)
const frozen = Object.freeze({ a: 1 });
frozen.a = 2; // silently fails (or throws in strict mode) – Object.freeze makes an object shallowly immutable
```

19. Destructuring — Array & Object

```
const user = { name: "Alex", age: 30, address: { city: "Mumbai" } };
const { name, age, address: { city } } = user;    // nested destructuring
const { name: userName } = user;                  // renaming while destructuring

const [first, second, ...rest] = [10, 20, 30, 40]; // array destructuring + rest
console.log(first, second, rest); // 10 20 [30, 40]

function printUser({ name, age = 18 }) { // destructuring directly in function parameters, with
  a default
    console.log(`${name} is ${age}`);
}
```

Real-time example: A React component receiving props typically destructures them directly in the function signature — `function ProductCard({ title, price, onAddToCart })` — dramatically more readable than repeatedly typing `props.title`, `props.price` throughout the component body.

20. Spread & Rest Operators Deep Dive

```
// Spread – EXPANDS an iterable into individual elements
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];           // [1, 2, 3, 4, 5] – new array, arr1 untouched
const merged = { ...defaults, ...userOverrides }; // shallow merge, later keys win on conflict
const copy = { ...original };           // a shallow COPY, not the same reference

// Rest – COLLECTS multiple elements into a single array/object (the opposite direction)
function sum(...nums) { return nums.reduce((a,b) => a+b, 0); }
const { id, ...otherFields } = user;    // otherFields = user minus the id property
```

Spread vs Rest — same `...` syntax, opposite direction: Spread EXPANDS a collection out (used when creating a new array/object/argument list); Rest COLLECTS remaining items INTO a collection (used in destructuring/function parameters). The direction/context tells you which one applies.

ISSUE `const copy = {...original}` "copy" still shares changes with the original when a NESTED object property is modified.

FIX Spread only creates a SHALLOW copy — nested objects/arrays inside are still shared by reference. Use `structuredClone(original)` or a deep-clone utility for genuinely independent nested copies.

21. Template Literals

```
const name = "Alex", age = 30;
const message = `Hello, ${name}! You are ${age} years old.`; // embedded expressions
const multiline = `Line one
Line two`; // real newlines, no \n needed
const html = `<div class="card">${renderTitle(title)}</div>`; // building markup strings
```

Before template literals, developers concatenated strings with `+`, which was error-prone with quotes/spacing and impossible to multi-line cleanly: `"Hello, " + name + "! You are " + age + " years old."` — template literals (backticks, ```, ``${}``) are strictly easier to read and maintain.

22. ES6+ Feature Recap Table

Feature	Introduced	Replaces / Improves
let/const	ES6 (2015)	var — adds real block scope
Arrow functions	ES6	function expressions — shorter, lexical "this"
Template literals	ES6	string concatenation with +
Destructuring	ES6	manual property-by-property extraction
Spread/Rest	ES6 (spread for arrays), ES2018 (object spread)	Object.assign, arguments object
Promises	ES6	raw callbacks (Section 28-29)
Classes	ES6	manual prototype-based inheritance syntax (Section 23-24)
Modules (import/export)	ES6	script tags + global variables, CommonJS require()
async/await	ES2017	chained .then() promise syntax (Section 30)
Optional chaining ?. / Nullish ??	ES2020	manual && chains and fallback pitfalls (Section 6)

23. Prototypes & Prototypal Inheritance Deep Dive

Unlike class-based languages (Java, C#), JavaScript's underlying inheritance model is **prototypal** — every object has an internal link to another object (its "prototype"), and property lookups walk UP this prototype chain until found (or reach `null`).

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound.`);
};

const dog = new Animal("Rex");
dog.speak(); // "Rex makes a sound." – speak() isn't ON dog itself, found via the prototype chain

console.log(dog.hasOwnProperty("name")); // true – own property
console.log(dog.hasOwnProperty("speak")); // false – inherited via prototype
console.log(Object.getPrototypeOf(dog) === Animal.prototype); // true
```

Why this matters: ES6 `class` syntax (Section 24) is NOT a new inheritance model — it's "syntactic sugar" that compiles down to exactly this same prototype-chain mechanism underneath. Understanding prototypes explains WHY methods defined on a class are shared efficiently across all instances (they live once on the prototype, not duplicated per instance) rather than being a mystery.

24. Classes in JavaScript

```
class Animal {
  constructor(name) {
    this.name = name;           // instance property, unique per object
  }
  speak() {                    // method – lives on Animal.prototype, shared by all
instances
    console.log(`${this.name} makes a sound.`);
  }
}
const dog = new Animal("Rex");
dog.speak(); // "Rex makes a sound."
```

Classes are simply a cleaner, more familiar SYNTAX over the exact same prototype mechanism from Section 23 — `class Animal { speak() {...} }` places `speak` on `Animal.prototype`, identical to manually writing `Animal.prototype.speak = function() {...}`.

25. Class Inheritance, super & Static Members

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound.`); }
  static createDefault() { return new Animal("Unnamed"); } // called on the CLASS, not an
instance
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // MUST call super() before using 'this' in a subclass
  }
  constructor
    this.breed = breed;
  }
  speak() { // overriding the parent method
    super.speak(); // still calls Animal's original version too
    console.log(`${this.name} barks!`);
  }
}

const rex = new Dog("Rex", "Labrador");
rex.speak(); // "Rex makes a sound." then "Rex barks!"

const defaultAnimal = Animal.createDefault(); // static method – called on the class itself
```

ISSUE "Must call super constructor before accessing 'this' or returning from derived constructor" error.

FIX In any subclass constructor, `super(...)` MUST be called before using `this` anywhere — JavaScript enforces this ordering to ensure the parent's initialization happens first.

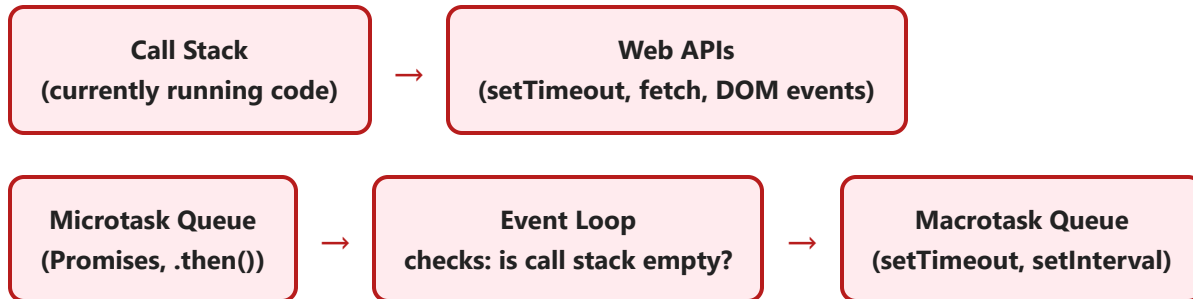
26. Getters & Setters

```
class Circle {
  constructor(radius) { this._radius = radius; }
  get area() { return Math.PI * this._radius ** 2; }    // accessed like a PROPERTY, not
called like a method
  set radius(value) {
    if (value <= 0) throw new Error("Radius must be positive");
    this._radius = value;
  }
}
const c = new Circle(5);
console.log(c.area);    // 78.53... – no parentheses! Looks like a plain property from outside
c.radius = 10;          // runs the setter's validation logic automatically
```

Getters/setters let you expose a clean PROPERTY-like interface while running real logic (computed values, validation) behind the scenes — useful for encapsulation without forcing every caller to remember to call a `.getArea()` method explicitly.

27. The Event Loop Deep Dive — Call Stack, Task Queue, Microtask Queue

JavaScript is **single-threaded** — it can only do ONE thing at a time. Yet it handles asynchronous operations (timers, network requests) without blocking. The **Event Loop** is the mechanism that makes this possible.



```

console.log("1: Start");

setTimeout(() => console.log("2: Timeout (macrotask)"), 0);

Promise.resolve().then(() => console.log("3: Promise (microtask)"));

console.log("4: End");

// ACTUAL OUTPUT ORDER: 1, 4, 3, 2
// Synchronous code (1, 4) always runs first, completing the call stack.
// THEN the microtask queue (Promises) is fully drained.
// ONLY THEN does the event loop pick up the next macrotask (setTimeout).
  
```

The single most important event loop rule for interviews: The microtask queue (Promises) is ALWAYS fully emptied before the event loop picks even ONE task from the macrotask queue (setTimeout/setInterval) — this is why a `Promise.resolve().then()` always logs before a `setTimeout(fn, 0)`, even though both are "asynchronous."

ISSUE A synchronous function containing a massive loop or heavy computation freezes the entire page — buttons stop responding, animations stutter.

FIX JavaScript is single-threaded — a long-running synchronous task **BLOCKS** the call stack entirely, preventing the event loop from processing ANY other events (clicks, renders) until it finishes. Break heavy work into smaller async chunks, use `requestIdleCallback`, or move genuinely heavy computation to a Web Worker (a separate thread).

28. Callbacks & Callback Hell

```
// "Callback Hell" – deeply nested callbacks become unreadable and hard to maintain
getUser(userId, function(user) {
  getOrders(user.id, function(orders) {
    getOrderDetails(orders[0].id, function(details) {
      getShipping(details.id, function(shipping) {
        console.log(shipping); // 4 levels deep and still growing – the "pyramid of
doom"
      });
    });
  });
});
```

This exact readability/maintainability problem — needing to chain many sequential async operations — is precisely what Promises (Section 29) and `async/await` (Section 30) were introduced to solve.

29. Promises Deep Dive

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: "Alex" });
      else reject(new Error("Invalid user ID"));
    }, 1000);
  });
}

fetchUser(1)
  .then(user => fetchOrders(user.id)) // chaining – each .then() gets the previous one's
  resolved value
  .then(orders => console.log(orders))
  .catch(error => console.error("Something failed:", error.message)) // catches ANY failure
  in the chain
  .finally(() => console.log("Done, regardless of outcome"));

// Running multiple promises concurrently
Promise.all([fetchUser(1), fetchUser(2)]) // waits for ALL – rejects immediately if ANY
one rejects
  .then(([user1, user2]) => console.log(user1, user2));

Promise.allSettled([fetchUser(1), fetchUser(-1)]) // waits for ALL, never rejects – gives status
of each
  .then(results => console.log(results));
```

Promise State	Meaning
Pending	The initial state — operation not yet complete
Fulfilled	Operation completed successfully — <code>.then()</code> handlers run
Rejected	Operation failed — <code>.catch()</code> handlers run

A Promise's chain flattens naturally — unlike callback hell, each `.then()` is a flat, sequential step rather than a deeper nested level, dramatically improving readability for sequential async operations.

ISSUE An error inside a `.then()` callback seems to "disappear" silently instead of being caught.

FIX Always add a `.catch()` at the end of every promise chain — an uncaught rejection anywhere in the chain propagates down to the nearest `.catch()`; without one, the rejection is silently swallowed (or logged as an "unhandled promise rejection" warning).

30. Async/Await Deep Dive

```
async function loadUserData(id) {
  try {
    const user = await fetchUser(id);          // "pauses" this function until the promise
    resolves
    const orders = await fetchOrders(user.id);  // reads top-to-bottom like synchronous
    code
    return { user, orders };
  } catch (error) {                            // catches rejections from ANY awaited call
    above
    console.error("Failed to load user data:", error.message);
    throw error;                               // re-throw if the caller also needs to
    handle it
  }
}

// Running independent async calls CONCURRENTLY instead of sequentially
async function loadBoth() {
  const [user, settings] = await Promise.all([fetchUser(1), fetchSettings()]);
  return { user, settings };
}
```

async/await is NOT a different async model — it's syntax sugar over Promises, making asynchronous code READ like synchronous code (top-to-bottom, with familiar try/catch) while still being fully non-blocking underneath.

ISSUE Two independent `await` calls run one after another, doubling the total wait time unnecessarily.

FIX If the two calls don't depend on each other's results, start both BEFORE awaiting either (or use `Promise.all`) so they run CONCURRENTLY instead of sequentially: `const [a, b] = await Promise.all([fetchA(), fetchB()]);` instead of `const a = await fetchA(); const b = await fetchB();`.

31. Error Handling — try/catch & Custom Errors

```
try {
  JSON.parse("{invalid json}");
} catch (error) {
  console.error("Parsing failed:", error.message);
} finally {
  console.log("This always runs, error or not");
}

class ValidationError extends Error { // custom error types – inherit from built-in Error
  constructor(message, field) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

function validateAge(age) {
  if (age < 0) throw new ValidationError("Age cannot be negative", "age");
}

try {
  validateAge(-5);
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Invalid field: ${error.field}`);
  } else {
    throw error; // re-throw anything we don't specifically know how to handle
  }
}
```

Custom Error subclasses let calling code distinguish DIFFERENT failure types (`instanceof ValidationError` vs a generic network error) and react appropriately, instead of parsing fragile error message strings to guess what went wrong.

32. Fetch API & Making HTTP Requests

```

async function getProducts() {
  const response = await fetch("/api/products");
  if (!response.ok) { // fetch does NOT reject on 4xx/5xx – you must
    check manually!
    throw new Error(`HTTP error: ${response.status}`);
  }
  const data = await response.json(); // parses the JSON response body
  return data;
}

async function createOrder(orderData) {
  const response = await fetch("/api/orders", {
    method: "POST",
    headers: { "Content-Type": "application/json", "Authorization": `Bearer ${token}` },
    body: JSON.stringify(orderData)
  });
  return response.json();
}

```

ISSUE A 404 or 500 error response is silently treated as a "success" by the calling code.

FIX `fetch()` ONLY rejects on genuine network failures (no connection at all) — it does NOT reject for HTTP error status codes like 404/500. Always explicitly check `response.ok` (or `response.status`) and throw/handle accordingly.

33. JSON Deep Dive

```

const user = { name: "Alex", age: 30 };
const jsonString = JSON.stringify(user); // '{"name":"Alex","age":30}' – object to string
const parsedBack = JSON.parse(jsonString); // back to a real JavaScript object

```

```

JSON.stringify(user, null, 2); // pretty-printed with 2-space indentation – great for
logging/debugging
JSON.stringify({ a: undefined, b: function() {}, c: 1 }); // '{"c":1}' – functions/undefined are
silently dropped!

```

JSON (JavaScript Object Notation) is a pure DATA format — it cannot represent functions, `undefined`, `Symbol`, or circular references (stringifying a circular object throws an error). This is exactly why it's the universal format for REST APIs (like the ones covered in the API Development guide) — it's simple, language-agnostic, and unambiguous.

34. The DOM Deep Dive — Selecting & Manipulating Elements

```
const card = document.querySelector(".card");           // first match
const allCards = document.querySelectorAll(".card");    // NodeList of ALL matches
const header = document.getElementById("header");

card.textContent = "New text";           // sets plain text (safe – no HTML parsing, avoids XSS)
card.innerHTML = "<b>Bold text</b>";    // parses as HTML – DANGEROUS with untrusted user input!

card.classList.add("active");
card.classList.remove("hidden");
card.classList.toggle("expanded");
card.style.backgroundColor = "yellow";

const newDiv = document.createElement("div");
newDiv.textContent = "I'm new!";
document.body.appendChild(newDiv);
```

ISSUE Setting `element.innerHTML` with unsanitized user-submitted content creates a Cross-Site Scripting (XSS) vulnerability.

FIX Use `textContent` for plain text (never parsed as HTML/scripts), or properly sanitize any HTML content through a trusted library (like DOMPurify) before ever assigning it via `innerHTML`.

35. Events Deep Dive — Bubbling, Capturing & Delegation

```
button.addEventListener("click", (event) => {
  console.log("Button clicked", event.target);
  event.stopPropagation(); // prevents the event from bubbling further up
});

// Event Delegation – ONE listener on a parent handles clicks on MANY current AND future children
document.querySelector(".product-list").addEventListener("click", (event) => {
  if (event.target.matches(".add-to-cart-btn")) {
    const productId = event.target.dataset.productId;
    addToCart(productId);
  }
});
```

Capturing Phase
(top → target)



Target
(the clicked element)



Bubbling Phase
(target → top)

Why event delegation matters: Attaching 200 individual click listeners to 200 product cards wastes memory and doesn't automatically work for cards added LATER (e.g., after infinite scroll loads more). ONE delegated listener on the shared parent handles ALL current and future children automatically, since it relies on event BUBBLING up to the parent rather than listening on each child directly.

ISSUE A dynamically-added button (inserted after page load) doesn't respond to clicks at all.

FIX A listener attached directly to the OLD button before it was replaced/re-rendered won't exist on the new element — use event delegation on a stable parent container instead of attaching listeners to individual dynamic elements.

36. Forms & Events in JavaScript

```
form.addEventListener("submit", (event) => {
  event.preventDefault();          // stops the browser's default full-page-reload form
  submission
  const formData = new FormData(form);
  const email = formData.get("email");
  // ... validate and send via fetch() instead of a native page reload
});

input.addEventListener("input", (event) => {
  console.log("Current value:", event.target.value); // fires on every keystroke
});
```

`event.preventDefault()` is essential for building a Single Page Application-style form submission — without it, the browser performs its OWN default full navigation/reload the moment the form submits, discarding your JavaScript-driven fetch logic entirely.

37. localStorage & sessionStorage in JavaScript

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");    // "dark"
localStorage.removeItem("theme");
localStorage.clear();                          // wipes everything

// localStorage only stores STRINGS — objects must be serialized
localStorage.setItem("user", JSON.stringify({ name: "Alex" }));
const user = JSON.parse(localStorage.getItem("user"));
```

ISSUE `localStorage.getItem("user")` returns the literal string `"[object Object]"` instead of usable data.

FIX `localStorage` can only store strings — an object was stored WITHOUT `JSON.stringify()` first, so JavaScript silently called `.toString()` on it. Always `JSON.stringify()` before storing and `JSON.parse()` after retrieving objects.

38. Debouncing & Throttling

```
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId); // cancel the previous pending call
    timeoutId = setTimeout(() => fn(...args), delay); // only actually runs after the user
    PAUSES
  };
}
searchInput.addEventListener("input", debounce((e) => {
  searchAPI(e.target.value); // only fires once the user stops typing for 300ms
}, 300));

function throttle(fn, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      fn(...args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit); // guarantees at most once per 'limit'
      ms
    }
  };
}
window.addEventListener("scroll", throttle(() => updateScrollProgress(), 100));
```

Technique	Behavior	Real Use Case
Debounce	Waits until activity STOPS for a pause, then runs once	Search-as-you-type API calls, form validation
Throttle	Runs at most once every fixed interval, REGARDLESS of how much activity occurs	Scroll position tracking, resize handlers, mouse-move effects

Real-time example: A search box calling the API on every single keystroke generated 15 requests for a 15-character query — debouncing the call to fire only 300ms after the user stops typing reduced it to exactly 1 API call per actual search intent.

39. Modules — import/export Deep Dive

```
// mathUtils.js
export function add(a, b) { return a + b; }      // named export
export const PI = 3.14159;
export default function multiply(a, b) { return a * b; } // default export – one per file

// app.js
import multiply, { add, PI } from './mathUtils.js';
import * as MathUtils from './mathUtils.js';      // import everything as one namespace
object
```

Named Export	Default Export
Multiple per file, imported with EXACT matching names (or aliased with <code>as</code>)	Only ONE per file, imported with ANY chosen name
<code>import { add } from '...'</code>	<code>import multiply from '...' (no braces)</code>

ES Modules (native `import / export`) replaced older patterns like CommonJS `require()` / `module.exports` (still used in plain Node.js) and IIFE-based module patterns — modules provide genuine file-level scoping, so variables don't leak into the global namespace by default, solving a real pre-ES6 pain point.

40. Generators & Iterators

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;          // "pauses" here, returning the current value, until next() is
    called again
  }
}

const gen = idGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2

function* range(start, end) {
  for (let i = start; i <= end; i++) yield i;
}

for (const num of range(1, 5)) console.log(num); // generators are directly usable with for...of
```

Generators are relatively advanced/niche in everyday app code, but they underpin async iteration patterns and are worth recognizing in interviews — a generator function (`function*`) can PAUSE and RESUME execution via `yield` , unlike a normal function which always runs start-to-finish in one go.

41. Map & Set Deep Dive

```
const map = new Map();
map.set("name", "Alex");
map.set(42, "numeric key allowed!");    // Map keys can be ANY type, unlike plain objects
                                         (string/symbol only)
console.log(map.get("name"));           // "Alex"
console.log(map.size);                  // 2
for (const [key, value] of map) { console.log(key, value); }

const set = new Set([1, 2, 2, 3, 3, 3]); // automatically deduplicates
console.log(set.size);                  // 3
console.log([...set]);                  // [1, 2, 3] – easy conversion back to an array
```

Use Case	Better Tool
Key-value pairs with NON-string keys, or needing guaranteed insertion order and easy size checking	Map (over a plain object)
Removing duplicates from an array	Set — <code>[...new Set(array)]</code> is the standard one-line deduplication idiom

42. Regular Expressions Basics

```
const emailPattern = /^[\\w.-]+@[\\w.-]+\\.\\w+$/;  
console.log(emailPattern.test("user@example.com")); // true  
  
const text = "Phone: 123-456-7890";  
const match = text.match(/\\d{3}-\\d{3}-\\d{4}/);  
console.log(match[0]); // "123-456-7890"  
  
const cleaned = "Hello   World".replace(/\\s+/g, " "); // collapses multiple spaces into one
```

Symbol	Meaning
\\d	Any digit (0-9)
\\w	Any word character (letters, digits, underscore)
\\s	Any whitespace character
+ / * / ?	One-or-more / zero-or-more / zero-or-one occurrences
g flag	Global — match ALL occurrences, not just the first

Regex is powerful but easy to get subtly wrong (especially for something as complex as fully-correct email validation) — for anything beyond simple patterns, prefer a well-tested library or the browser's native `<input type="email">` validation (HTML/CSS guide, Section 12) over hand-rolling a complex regex.

43. Memory Management & Garbage Collection

JavaScript automatically manages memory via **garbage collection** — you never manually "free" memory like in C/C++. The engine periodically identifies objects no longer REACHABLE from any active reference (variables, closures, the global scope) and reclaims their memory.

Mark-and-sweep, simplified: The garbage collector starts from "roots" (global variables, currently executing function scopes) and marks every object reachable from there. Anything NOT marked after this walk is considered garbage and gets swept away, freeing its memory.

44. Memory Leaks — Common Causes & Fixes

ISSUE

A single-page application gets progressively slower and consumes more memory the longer the user stays on it.

FIX

Almost always an accumulating memory leak — profile with browser DevTools' Memory tab (take heap snapshots before/after an action, compare) to find which objects keep growing in count.

ISSUE

Detached DOM elements remain in memory long after being removed from the visible page.

FIX

An event listener or a closure still holds a reference to the removed element — always call `removeEventListener` (or use an `AbortController`) when a component/element is destroyed, matching every `addEventListener` with a corresponding cleanup.

ISSUE

A `setInterval` timer keeps running and referencing old data even after the component/page that created it is gone.

FIX

Always store the interval/timeout ID and call `clearInterval` / `clearTimeout` during cleanup (e.g., a React `useEffect` cleanup function) — forgetting this is one of the most common real memory leak sources in SPAs.

ISSUE

A global cache/array keeps growing indefinitely and is never cleared.

FIX

Any accumulating global collection (a cache, a log buffer) needs an explicit eviction/size-limit strategy (e.g., an LRU cache) — unbounded global growth is a memory leak even though nothing is technically "broken."

45. Strict Mode

```
"use strict";

x = 10; // ReferenceError in strict mode (would silently create a global variable without it!)

function doSomething() {
  "use strict"; // can also be applied per-function
}
```

Strict mode catches common mistakes early (accidental globals, assigning to read-only properties, duplicate parameter names) by turning silent failures into real errors. ES6 modules and class bodies are AUTOMATICALLY in strict mode — you rarely need to add the directive manually in modern code.

46. Immutability & Pure Functions

```
// Impure – depends on/modifies external state, unpredictable given the same input
let total = 0;
function addToTotal(n) { total += n; return total; }

// Pure – same input ALWAYS produces the same output, no side effects
function add(a, b) { return a + b; }

// Immutable update pattern (critical in React state management)
function addItem(cart, item) {
  return [...cart, item]; // returns a NEW array – never mutates the original 'cart'
}
```

Why this matters in real frameworks: React (and similar libraries) detect changes by comparing OLD vs NEW references — if you mutate an array/object directly instead of creating a new one, React may not detect the change at all and fail to re-render, a very common real-world bug category.

47. Functional Programming Concepts in JavaScript

Concept	Meaning	Example
First-class functions	Functions can be assigned to variables, passed as arguments, returned from other functions	<pre>const handler = () => {...}; button.onclick = handler;</pre>
Higher-order functions	A function that takes another function as an argument or returns one	<pre>array.map(fn), debounce(fn, delay)</pre>
Pure functions	No side effects, deterministic output for a given input (Section 46)	<pre>add(a, b)</pre>
Immutability	Never mutating existing data, always producing new data (Section 46)	Spread-based array/object updates
Function composition	Combining small functions into a larger pipeline	<pre>const process = (x) => formatCurrency(round(calculateTotal(x)));</pre>

48. Common Design Patterns in JavaScript

```
// Module Pattern (using a closure to create private state)
const CounterModule = (function() {
  let count = 0; // private - inaccessible from outside
  return { increment: () => ++count, getCount: () => count };
})();

// Observer Pattern (the foundation of event systems, including React state updates)
class EventEmitter {
  constructor() { this.listeners = {}; }
  on(event, callback) {
    (this.listeners[event] || []).push(callback);
  }
  emit(event, data) {
    (this.listeners[event] || []).forEach(cb => cb(data));
  }
}

// Singleton Pattern (ensuring only one instance exists)
class Database {
  static instance;
  static getInstance() {
    if (!Database.instance) Database.instance = new Database();
    return Database.instance;
  }
}
```

49. Performance Optimization Tips

- **Debounce/throttle** expensive handlers tied to frequent events (scroll, resize, input) — Section 38.
- **Avoid unnecessary re-renders** in frameworks by keeping state updates immutable and minimal (Section 46).
- **Lazy-load** code/images not needed immediately (dynamic `import()`, HTML's `loading="lazy"` from the HTML/CSS guide).
- **Batch DOM updates** — reading layout properties (`offsetHeight`) repeatedly interleaved with writes forces multiple expensive reflows ("layout thrashing"); group all reads, then all writes.
- **Use Web Workers** for genuinely heavy CPU computation, keeping the main thread free to respond to user interaction.

50. NPM & package.json Deep Dive

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.0"      // needed in PRODUCTION
  },
  "devDependencies": {
    "jest": "^29.0.0"        // needed only for DEVELOPMENT/testing, not shipped to production
  }
}
```

```
npm install          # installs everything from package.json into node_modules
npm install axios    # adds a new dependency, updates package.json automatically
npm install --save-dev jest # adds to devDependencies specifically
npm run start        # runs the named "start" script
```

`package-lock.json` pins the EXACT resolved version of every dependency (and its own transitive dependencies) actually installed — committing it ensures every developer and CI server installs the identical dependency tree, avoiding "works on my machine" bugs from subtly different sub-dependency versions, directly analogous to Maven's dependency management.

51. Modern Build Tools — Webpack & Vite Overview

Tool	What It Does
Webpack	Bundles many JS/CSS/asset files into optimized production bundles; highly configurable but historically slower dev-server rebuild times
Vite	Modern build tool leveraging native browser ES modules during development for near-instant hot-reload, using Rollup for optimized production bundles
Babel	Transpiles modern JS syntax (optional chaining, newer features) down to older syntax older browsers can still run

Why bundlers are needed at all: browsers historically couldn't efficiently load hundreds of separate small module files over the network — a bundler combines/minifies/optimizes everything into a small number of production-ready files, plus handles transpiling modern syntax for broader browser support.

52. Testing JavaScript — Jest Basics

```
function add(a, b) { return a + b; }

test('adds 1 + 2 to equal 3', () => {
  expect(add(1, 2)).toBe(3);
});

describe('Cart', () => {
  test('adding an item increases cart size', () => {
    const cart = [];
    const updated = addItem(cart, { id: 1 });
    expect(updated).toHaveLength(1);
    expect(cart).toHaveLength(0); // verifying the ORIGINAL array wasn't mutated (Section
46)
  });
});
```

Testing pure functions (Section 46) is dramatically simpler than testing functions with side effects — no mocking required, just assert output for a given input, exactly why functional purity is valued in real, well-tested codebases.

53. JavaScript in a Full-Stack World (React + Spring Boot)



Every concept in this guide directly powers real React development: `useState` / `useEffect` rely on closures (Section 13) and immutable updates (Section 46); API calls use `fetch`/`async-await` (Sections 30-32); JSX event handlers rely on correct "this"/arrow-function behavior (Section 14); and `npm/package.json` (Section 50) manages every dependency React itself is built from — this guide's fundamentals are the direct foundation for the React + Spring Boot API Integration guide's patterns.

54. Common JavaScript Bugs, Issues & Fixes (Extended)

ISSUE "Cannot read properties of undefined (reading 'x')" — the most common JavaScript runtime error.

FIX Accessing a property on something that's actually `undefined` / `null` — use optional chaining (`obj?.x`, Section 6) or verify the data exists (e.g., after an async fetch resolves) before accessing nested properties.

ISSUE A number calculation gives an unexpected result like `0.1 + 0.2 = 0.30000000000000004`.

FIX Standard IEEE-754 floating-point precision limitation, not a JavaScript-specific bug — for money/currency, work in integer cents/paise, or use a decimal library, never raw floating-point arithmetic for exact financial values.

ISSUE An infinite loop or runaway recursive function freezes/crashes the browser tab.

FIX Verify the loop's exit condition is actually reachable, and that recursive functions have a correct, reachable base case — add a maximum iteration/depth safety guard during development to catch this faster.

ISSUE State/UI doesn't update even though the underlying array/object was clearly changed in code.

FIX The data was **MUTATED** directly instead of replaced with a new reference (Section 46) — frameworks like React compare references to detect changes; always create a new array/object copy for updates.

ISSUE An async function's result is used before it has actually resolved, producing `undefined` or a Promise object instead of real data.

FIX Missing an `await` (or a `.then()`) — remember that ANY function marked `async` always returns a Promise, which must itself be awaited/chained wherever its result is used.

ISSUE A "this is undefined" error inside a class method used as a callback (e.g., `<button onClick={this.handleClick}>`).

FIX Passing a method as a bare reference loses its "this" binding (Section 14) — bind it in the constructor (`this.handleClick = this.handleClick.bind(this)`), or define the method as an arrow function class field instead.

ISSUE Comparing two arrays or objects with `===` always returns false even when they "look" identical.

FIX `===` compares REFERENCE identity for objects/arrays, not deep content — use a deep-equality check (e.g., `JSON.stringify(a) === JSON.stringify(b)` for simple cases, or a proper deep-equal utility for anything with nested dates/functions/undefined values).

55. JavaScript Interview Questions & Answers (40 Q&A)

Q1. What's the difference between var, let, and const?

A: var is function-scoped, hoisted and initialized to undefined, and can be redeclared. let/const are block-scoped, hoisted but NOT initialized (Temporal Dead Zone), and cannot be redeclared. const additionally cannot be reassigned (though its contents can still be mutated).

Q2. What is a closure?

A: A function that retains access to variables from its enclosing scope even after that outer function has finished executing — used for data privacy, function factories, and correctly capturing per-iteration values in loops.

Q3. Explain the event loop and why setTimeout(fn, 0) doesn't run immediately.

A: JavaScript is single-threaded; the event loop only processes queued callbacks once the call stack is empty. setTimeout(fn, 0) still queues its callback in the macrotask queue, which only runs AFTER all currently synchronous code (and the entire microtask/Promise queue) has finished.

Q4. What's the difference between == and ===?

A: == performs type coercion before comparing (often producing surprising results); === compares both value and type with no conversion — always prefer === in real code.

Q5. What is "this" in a regular function vs an arrow function?

A: In a regular function, "this" depends on HOW the function is called (the object before the dot, or undefined/global if called standalone). Arrow functions don't have their own "this" at all — they inherit it lexically from the enclosing scope at the time they were defined.

Q6. What's the difference between null and undefined?

A: undefined means a variable has been declared but never assigned a value (JavaScript's own default); null is an explicit value a developer assigns to intentionally represent "no value" — a meaningful distinction in APIs distinguishing "not yet loaded" from "loaded, but empty".

Q7. How do you deep clone an object in JavaScript?

A: Use `structuredClone(obj)` (modern, handles most cases including dates/Maps/Sets), or for simple JSON-safe data, `JSON.parse(JSON.stringify(obj))` — `spread ( {...obj} )` only performs a SHALLOW copy.

Q8. What's the difference between map() and forEach()?

A: map() returns a NEW array containing the transformed values; forEach() returns undefined and is used purely for side effects (logging, mutating external state) — using forEach when you actually need a new array is a common code-smell.

Q9. Explain prototypal inheritance.

A: Every JavaScript object has an internal link to another object (its prototype); property/method lookups that

fail on the object itself automatically continue up this prototype chain until found or the chain ends at null — the underlying mechanism ES6 classes compile down to.

Q10. What is a Promise, and what are its three states?

A: An object representing the eventual completion (or failure) of an asynchronous operation. Its three states are pending (not yet resolved), fulfilled (succeeded), and rejected (failed) — once settled (fulfilled or rejected), a Promise's state never changes again.

Q11. What does `async/await` actually do under the hood?

A: It's syntax sugar over Promises — an `async` function always returns a Promise, and `await` pauses execution of that function (without blocking the rest of the program) until the awaited Promise settles, making asynchronous code readable in a synchronous top-to-bottom style.

Q12. Why does `fetch()` not reject on a 404 or 500 response?

A: `fetch()` only rejects for genuine network-level failures (e.g., no connectivity, DNS failure); an HTTP error status code is still considered a "successful" fetch from the network's perspective, so you must manually check `response.ok` / `response.status` and handle errors explicitly.

Q13. What is event delegation and why is it useful?

A: Attaching a single event listener to a shared parent element instead of individual listeners on many children, relying on event bubbling — it's more memory-efficient at scale and automatically works for child elements added dynamically after the listener was set up.

Q14. What's the difference between `debounce` and `throttle`?

A: `Debounce` waits for activity to fully STOP for a pause before running once (good for search-as-you-type); `throttle` guarantees the function runs at most once per fixed time interval regardless of ongoing activity (good for scroll/resize handlers).

Q15. How would you avoid a classic `var`-based closure bug inside a `for` loop?

A: Replace `var` with `let` — `let` creates a fresh, independent binding of the loop variable for each iteration, correctly captured by any closures (like `setTimeout` callbacks) created inside that iteration.

Q16. What is the difference between `call`, `apply`, and `bind`?

A: `call(thisArg, arg1, arg2)` invokes a function immediately with a given "this" and individual arguments; `apply(thisArg, [args])` does the same but takes arguments as an array; `bind(thisArg)` returns a NEW function permanently bound to that "this", without invoking it immediately.

Q17. Why should you use `===` instead of `==` in production code?

A: `==` applies often-confusing implicit type coercion rules (e.g., `"" == false` is true) that are a common source of subtle bugs; `===` never performs coercion, making comparisons predictable and explicit.

Q18. What causes a memory leak in a long-running single-page application?

A: Common causes include event listeners never removed after their element is destroyed, timers (setInterval) never cleared, and ever-growing global caches/arrays with no eviction strategy — all keeping objects artificially "reachable" so the garbage collector can never reclaim them.

Q19. What is the difference between synchronous and asynchronous code in JavaScript?

A: Synchronous code runs immediately, in order, blocking further execution until it completes. Asynchronous code (timers, network calls) is handed off to the runtime/Web APIs and its callback/continuation runs LATER via the event loop, without blocking the rest of the program in the meantime.

Q20. What is destructuring, and why is it useful?

A: A syntax for extracting multiple values from arrays/objects into individual variables in one concise statement — improves readability significantly, especially for function parameters (like React component props) and API response handling.

Q21. What's the difference between Object.freeze() and const?

A: const only prevents REASSIGNING the variable itself — the object's properties remain fully mutable. Object.freeze() actually prevents modifying, adding, or removing the object's OWN properties (shallowly — nested objects inside are still mutable unless also frozen).

Q22. How do JavaScript modules (import/export) improve on older global-script patterns?

A: Each module has its own genuine file-level scope by default — variables don't leak into the global namespace unless explicitly exported, solving the pre-ES6 problem of scripts polluting one shared global scope and risking naming collisions.

Q23. What is a pure function, and why does it matter for state management (e.g., in React)?

A: A function that always returns the same output for the same input, with no side effects. Pure, immutable state updates let frameworks reliably detect changes via simple reference comparison, and make the resulting code far easier to test and reason about.

Q24. Explain the difference between shallow copy and deep copy.

A: A shallow copy (spread, Object.assign) duplicates only the top-level properties — any NESTED objects/arrays inside are still shared by reference with the original. A deep copy (structuredClone) recursively duplicates every nested level, producing a fully independent structure.

Q25. What does "JavaScript is single-threaded but non-blocking" actually mean?

A: Only one line of JS code executes at any given instant (single-threaded), but slow operations (network calls, timers) are delegated to the browser/Node runtime's own Web APIs, which notify the event loop once complete — allowing the main thread to keep handling other work in the meantime instead of blocking on the slow operation.

Q26. Why is `NaN === NaN` false, and how do you correctly check for NaN?

A: By IEEE-754 floating-point specification, NaN is defined as never equal to anything, including itself — use `Number.isNaN(value)` (not the global, coercing `isNaN()`) to correctly and safely check for NaN.

Q27. What is the purpose of the "use strict" directive?

A: It enables a stricter parsing/execution mode that turns several silent mistakes (accidental global variable creation, assigning to a read-only property) into real thrown errors, catching bugs earlier — ES6 modules and classes are automatically strict by default.

Q28. How would you make an API call that must complete within 5 seconds or be aborted?

A: Use an `AbortController` — pass its signal to `fetch`'s options, and call `controller.abort()` after a 5-second `setTimeout`; `fetch` will then reject with an `AbortError`, which you catch and handle as a timeout.

Q29. What's the real difference between `Map` and a plain `Object` for storing key-value data?

A: `Map` allows keys of ANY type (not just strings/symbols), preserves insertion order reliably, provides a direct `.size` property, and doesn't have inherited prototype properties that could collide with user data — often the better, more predictable choice for pure dictionary-style data.

Q30. What is function composition, and how does it relate to functional programming?

A: Combining several small, single-purpose pure functions into a larger pipeline where each function's output feeds the next function's input — a core functional programming technique for building complex behavior from simple, independently testable pieces.

Q31. What is the difference between a macrotask and a microtask in the event loop?

A: Microtasks (`Promise` callbacks, `queueMicrotask`) are ALWAYS fully drained before the event loop picks up the next macrotask (`setTimeout`, `setInterval`, I/O callbacks) — this is why `Promise`-based code consistently runs before an equivalent `setTimeout(fn, 0)`.

Q32. Why might a `React` (or similar) app fail to re-render after updating an array in state?

A: The array was likely MUTATED in place (e.g., via `push()`) instead of replaced with a new array reference (via `spread/concat`) — frameworks typically detect changes by comparing references, so a mutated-but-same-reference array appears unchanged.

Q33. What's the difference between a named export and a default export in ES modules?

A: A file can have multiple named exports, imported using the EXACT matching name (or an alias); a file can have only ONE default export, importable under any name the importer chooses — named exports are generally preferred for utility modules since they're more explicit and support better tooling/autocomplete.

Q34. How would you safely access a deeply nested property that might not exist, without a series of manual null checks?

A: Use optional chaining: `const city = user?.address?.city;` — it short-circuits to `undefined` the moment any link in the chain is null/undefined, instead of throwing a `TypeError`.

Q35. What's a real-world example of the Observer pattern in JavaScript?

A: The DOM's own event system (`addEventListener/dispatchEvent`) is a built-in Observer pattern implementation; many state-management libraries (Redux, RxJS) and custom `EventEmitter` classes implement the same core idea — subscribers registering callbacks that get notified when a specific event/state change occurs.

Q36. Why should you avoid using array index as a React key when the list can be reordered/filtered?

A: (Cross-reference concept) Using index as a stable identity breaks when items are reordered/removed, since the same index now refers to a different logical item — frameworks rely on a STABLE, unique key tied to the actual data to correctly track which DOM element corresponds to which data item across re-renders.

Q37. What is the difference between `Array.prototype.slice()` and `Array.prototype.splice()`?

A: `slice()` is non-mutating — returns a new array without touching the original. `splice()` mutates the original array in place, removing and/or inserting elements, and separately returns an array of any removed elements.

Q38. How does JavaScript's garbage collector decide what memory to free?

A: Using a mark-and-sweep algorithm — starting from root references (global scope, active call stack), it marks every object still reachable; anything left unmarked afterward is unreachable "garbage" and its memory is reclaimed.

Q39. What's the benefit of using `async/await` over raw `Promise.then()` chains?

A: It reads top-to-bottom like ordinary synchronous code (including familiar `try/catch` error handling), which is generally easier to follow than long `.then()` chains, especially once conditional logic or loops are involved in the `async` flow.

Q40. Describe a real production bug you'd expect from mixing `var` and closures inside an `async` loop, and how to fix it.

A: Firing off several `async` operations inside a `var`-based `for` loop, each expecting to use "their" loop index once the `async` work completes, all incorrectly reference the SAME final index value once the loop has finished — fixed by switching to `let`, which gives each iteration its own independent, correctly-captured binding.

56. Quick Revision Summary

- Use `const` by default, `let` when reassignment is genuinely needed, never `var` in new code.
- Always use `===` / `!==` — never `==` / `!=` — to avoid confusing type-coercion bugs.
- Closures let functions "remember" their creation scope — the foundation of private state, factories, and correct loop-variable capture (with `let`).
- The event loop always fully drains the microtask (Promise) queue before processing the next macrotask (`setTimeout`).
- Promises/`async-await` replace callback hell with flat, readable sequential `async` code — always handle errors with `.catch()`/`try-catch`.
- Arrays/objects copy by REFERENCE — use `spread/structuredClone` for independent copies, and never mutate state directly in frameworks like React.
- ES6 classes are syntax sugar over the same underlying prototype-chain inheritance mechanism.
- Always clean up event listeners/timers to avoid memory leaks in long-running single-page apps.
- Debounce for "wait until activity stops" (search boxes); throttle for "run at most once per interval" (scroll/resize).
- `fetch()` only rejects on network failure — always manually check `response.ok` for HTTP error codes.